Performance Modeling of a KV-Stationary Systolic Array for Multi-Query Attention Inference

Final Report

Hudson Strauss Dhruv Patel

June 7, 2026

High-Performance Computer Architecture and Systems,
Santa Clara University, CSEN 318, Spring 2026

1 Introduction

Motivation. Transformer inference is increasingly bottlenecked by memory bandwidth as context windows grow (Dao et al., 2022). During autoregressive decoding, each generated token reloads the entire key-value (KV) cache from DRAM. Prefill is worse: a standard two-GEMM attention implementation writes and re-reads a full $H \times T \times T$ score matrix between passes, reaching **17 GB** at T=8,192, H=64, FP16.

Objective. This project models and evaluates a *KV-stationary* 2D systolic-array mapping for Multi-Query Attention (MQA) (Ainslie et al., 2023): keys and values are loaded once into array columns and remain resident while query vectors stream left-to-right, computing an online softmax inline at every column. The model is validated against SCALE-Sim (Samajdar et al., 2020) cycle-accurate simulation for the lower-MAC $Q \cdot K$ compute component. We sweep sequence length T, merge-extension depth n, and interleaved-MAC count lmc, comparing against both an unfused and a FlashAttention-style fused baseline on equal silicon.

Literature and market review. The dominant software approach to the score-matrix bottleneck is FlashAttention (Dao et al., 2022), which tiles the $Q \cdot K^\top \cdot V$ computation to keep intermediate scores on-chip. Hardware-level attention acceleration has focused primarily on sparse attention: A^3 (Ham et al., 2020) approximates the attention score distribution to skip low-weight tokens, and SpAtten (Wang et al., 2021) prunes tokens and heads cascade-style. Both require structured sparsity and do not eliminate the two-GEMM latency for dense contexts. SCALE-Sim (Samajdar et al., 2020) models dense GEMM throughput cycle-accurately but does not natively model single-pass streaming attention with inline online softmax. We did not identify prior work modeling a KV-stationary dataflow in which query vectors stream through stationary KV columns with per-column softmax accumulation.

2 System Design & Implementation Details

2.1 Design Evolution

The architecture evolved through four iterations, each correcting a measurable inefficiency.

Iteration 1 — 1D KV-stationary pipeline. Each PE stores one (K_i, V_i) pair; queries stream left-to-right and accumulate an online softmax, eliminating score-matrix DRAM (Figure 4). Two problems emerge: (i) *pipeline drain* — at T=8,192 the last packet must still traverse all remaining columns, consuming essentially 100% of runtime; (ii) the upper Running Attention PE needs only 7 cycles per score against a 128-cycle dot product, giving $\approx$5% utilization.

Iteration 2 — 2D multi-head extension. Assigning each of the H=64 heads its own row lane improves throughput $H\times$. MQA's shared KV cache means all rows read the same column data, so KV is loaded from DRAM once regardless of head count — but drain and 5% utilization remain.

Iteration 3 — Merge extensions. Stacking 2^n sub-arrays in the row dimension reduces per-query traversal to $T/2^n$ columns; a binary merge tree (Figures 6, 7) recombines partial (m, l, O) states with no post-pipeline overhead. With n=3 traversal drops 8$\times$ at the cost of 8$\times$ more rows (PE count unchanged), but gain is only realized in the compute-bound regime (§3.1).

Iteration 4 — Interleaved MACs (lmc). Time-interleaving lmc independent MACs per column, each staggered by $\lceil d/lmc \rceil$ cycles, raises upper-PE utilization to 87.5% at lmc=16 with no extra PE rows. Effective lmc is bounded by bandwidth: at commodity 512 B/cycle the balance point $lmc \approx$BW/128=4 is optimal; higher values help only with HBM-class bandwidth (§3.1).

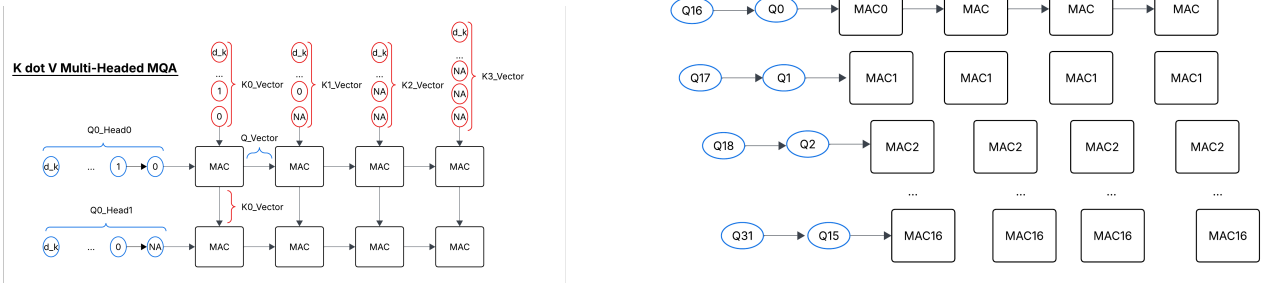
2.2 Array Layout and PE Structure

The physical array has $H{\cdot}2^n$ rows (parallel query lanes) by $T/2^n$ columns (KV-stationary token positions). Detailed diagrams of the single-row dataflow, Running Attention PE, and merge unit are in Appendix B (Figures 4–7).

Per-column PE (two stages). Stage 1: lmc interleaved serial MAC units compute $s_i{=}Q{\cdot}K_i$ over d cycles. With $lmc{>}1$, packets time-multiplex, each staggered by $\lceil d/lmc \rceil$ cycles. Stage 2 (Upper Running Attention PE): receives score s_i and running state (m, l, O) from the left neighbour and executes the online softmax update:

$$m' = \max(m,\, s_i), \quad l' = l\, e^{m-m'} + e^{s_i-m'}, \quad O' = O\, e^{m-m'} + V_i\, e^{s_i-m'}.$$

Cost: $\tau_{\text{exp}} + \lceil 3d/w \rceil = 4{+}3 = 7$ cycles. A shallow score buffer paces delivery at eff_stagger = max(packet_stagger, 7) to prevent state pile-up. After the final column: Output $= O_N/l_N$.



(a) Multi-head MQA layout. H row lanes share one set of KV columns; KV is loaded from DRAM exactly once regardless of head count.

(b) $lmc{=}16$ time-interleaved MACs. 16 query packets share one column staggered by 8 cycles; upper-PE utilization rises to 87.5%. Throughput ceiling: $lmc{=}21$ ($\lceil 128/21 \rceil{=}7$ exactly matches the 7-cycle upper-PE cost); beyond this the score buffer activates but no further throughput is gained.

Figure 1: KV-stationary array. Queries stream left-to-right; KV is stationary; online softmax runs inline, eliminating the $H{\times}T^2$ score-matrix DRAM round-trip.

2.3 Pipeline Timing Model

Because every column owns a *dedicated* lower MAC and stationary K_i, a packet's C dot products $s_0 \ldots s_{C-1}$ are independent and compute in a pipelined wavefront as Q ripples right. The $d{=}128$ fill is paid *once*; thereafter the binding path is the online-softmax state chain. For R active rows, C active columns, P packets per row:

$$\text{tile\_cycles} = \underbrace{d}_{\text{fill (once)}} + (R + C - 1) \cdot \text{eff\_stagger} + (P - 1) \cdot \text{eff\_stagger}.$$

SCALE-Sim measures $\sim$2.5 cycles/column of fill, confirming the wavefront model (§3.1).

DRAM traffic. At $T{=}8{,}192$: Q reads + KV load + O writes $\approx$ 273 MB (FP16), identical to a fused FlashAttention baseline. An *unfused* baseline adds the score matrix ($H{\times}T^2{\times}$bpe = 17.18 GB written then re-read = 17.18 GB), bringing total unfused DRAM to 17.45 GB of which the score matrix is 98.4%. Arithmetic intensity (MACs/byte) at $T{=}8{,}192$: unfused baseline 63 (compute-bound); fused FlashAttention 4,033 (memory-bound); KV-stationary 8,066 (memory-bound, 2$\times$ higher due to the additional $3d$ V-accumulation MACs per token pair). The identical DRAM traffic of the latter two explains why their cycle counts converge when both are memory-bound.

2.4 Implementation

The model is implemented in Python with five key files: `kv_stationary_model.py` (analytical pipeline, wavefront fill model, multi-pass re-use extension), `baseline_mqa_model.py` (roofline baseline, fused/unfused), `validate_lower_mac_sweep.py` and `validate_sweep_scalesim.py` (SCALE-Sim harness), and `reuse_full_sweep` (750-row bandwidth×lmc×T×P sweep). SCALE-Sim validates the $Q{\cdot}K$ GEMM at infinite bandwidth to isolate compute; the analytical model adds a roofline DRAM floor:

$$\begin{aligned} \text{corrected\_cyc} &= \text{pipeline\_cyc} - \text{analytical\_mac} + \text{ss\_mac\_cyc}, \\ \text{total\_cyc} &= \max(\text{corrected\_cyc},\ \text{DRAM\_cyc}). \end{aligned}$$

3 Experimental Results and Evaluation

Setup. Fixed hardware: H=64, d=128, FP16 (2 B/elem), DRAM BW = 512 B/cycle, upper-PE width w=128, τ_{exp}=4 cycles. Sweep: $T \in \{128, 512, 1024, 2048, 4096, 8192\}$; $n \in \{0, 3\}$; $lmc \in \{1, 2, 4, 8, 16\}$; prefill ($Q_T$=$T$). SCALE-Sim ran on 40 prefill configurations fitting the 500 MB operand-matrix limit; 9 large configurations (n=0, lmc=16, $T \geq 4096$) are analytical only. Decode validation failed (analytical diverges thousands of percent from SCALE-Sim because the model counts only d MAC cycles while SCALE-Sim measures the full pipeline traversal); decode results are excluded.

3.1 SCALE-Sim Validation and the Wavefront Fill

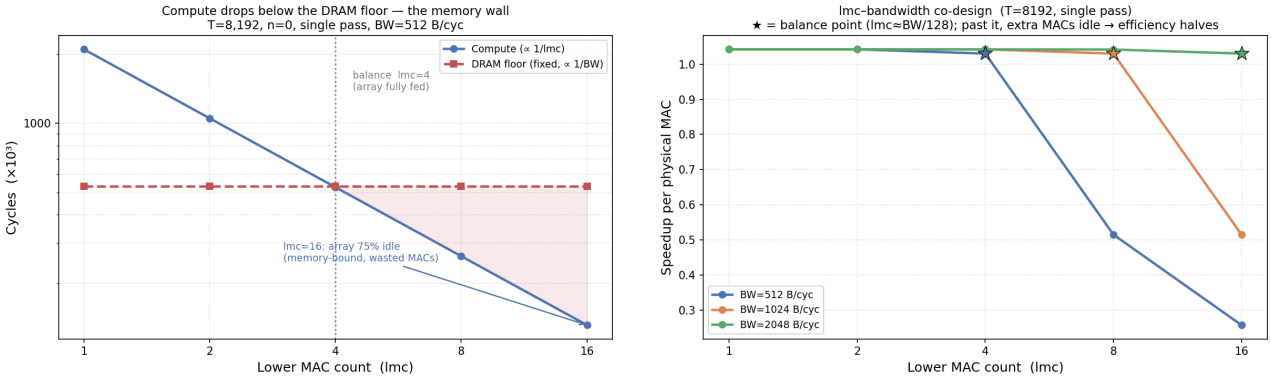
Table 1 shows representative results; δ% is the SCALE-Sim excess over the analytical MAC bound — the pipeline fill physically observed, *not* a modeling error. Across all 40 runs, fill clusters at $\sim$2.5 cycles/column regardless of lmc, confirming the wavefront fill model. Under the wavefront fill, prefill is DRAM-bound at every tested configuration.

Table 1: Lower-MAC SCALE-Sim validation (selected prefill). δ% is the measured wavefront fill as a fraction of the analytical MAC bound. Errors below 10% at $T \geq 1024$ show the model is accurate for large T.

T	n	lmc	Analytical	SCALE-Sim	δ%
128	0	1	16,384	16,891	3.1%
512	0	1	65,536	66,811	1.9%
512	0	16	4,096	5,371	31.1%
2048	0	16	16,384	20,731	26.5%
2048	3	16	16,384	17,147	4.7%
8192	3	16	65,536	67,835	3.5%

3.2 Bandwidth Co-Design: Sizing lmc to Bandwidth

With the validated wavefront fill, compute at T=8,192 drops to $\sim$132K cycles for lmc=16 — far below the DRAM floor of $\sim$525K cycles at 512 B/cycle. Compute scales as $1/lmc$ while the DRAM floor is fixed by bandwidth, so they cross at $lmc \approx \text{BW}/128$ (Figure 2). Below this balance point extra MACs sit idle: at lmc=16 on 512 B/cycle, 75% of multipliers are stalled, and per-MAC efficiency is only 0.26×. *This inverts the earlier design recommendation*: $lmc \leq 4$ is the efficient choice at 512 B/cycle; lmc=16 is only justified with $\sim$2 TB/s (HBM-class) bandwidth, where it runs 4× faster.



(a) Compute ($\propto 1/lmc$) crosses the DRAM floor at $lmc \approx$ BW/128; past it, MACs idle.

(b) Per-MAC efficiency vs lmc per bandwidth tier. The optimum ($\star$) marches right as bandwidth grows.

Figure 2: lmc–bandwidth co-design at T=8,192. At commodity 512 B/cycle the balance point is $lmc \approx 4$.

3.3 Prefill: KV-stat vs FlashAttention on Equal Silicon

Table 2 compares KV-stat against a fused FlashAttention baseline running on a single $H \times H = 64 \times 64$ array. Both eliminate score-matrix DRAM. *Raw* speedup uses this single-array Flash cycle count directly. *Per-MAC* = raw speedup ÷ physical-MAC ratio (KV_MACs/Flash_MACs), which adjusts for the fact that KV-stat deploys many more multipliers; above 1.0× means KV-stat delivers more throughput per multiplier.

Table 2: Prefill: KV-stat vs fused FlashAttention (wavefront model). DRAM-bound = 512 B/cycle; Compute-only = no memory wall (SCALE-Sim regime). † = analytical (operand-limit exceeded).

		DRAM-bound		Compute-only		
Config	T	Raw	per-MAC	Raw	per-MAC	MAC ratio
n=0, lmc=1	512	7.9×	1.00×	7.9×	1.00×	8×
n=0, lmc=1	8192	133×	1.04×	133×	1.04×	128×
n=0, lmc=16	512	33×	0.26×	124×	0.97×	128×
n=0, lmc=16	8192†	527×	0.26×	2133×	1.04×	2048×
n=3, lmc=16	2048	134×	0.26×	775×	1.51×	512×
n=3, lmc=16	8192	535×	0.26×	3604×	**1.76**×	2048×

Three findings stand out. **(1)** At 512 B/cycle, lmc=16 sits at 0.26× per MAC (75% idle); lean lmc=1 stays compute-bound at ∼1.0×. **(2)** Merge extensions (n=3) are *inert* when DRAM-bound — they cut column traversal (a compute term) without touching DRAM, so n=0 and n=3 lock to the same floor. **(3)** Compute-only, merge extensions *return*: n=3, lmc=16 reaches **1.76**× per MAC at T=8,192, where the taller array's shorter traversal dominates. Cycle breakdowns and equal-area speedup curves across all four configurations in the compute-bound regime are in Appendix C (Figures 8–11).

3.4 Comparison Against Unfused Baseline

Table 3: Two-way speedup: KV-stat (n=3, lmc=16) vs unfused baselines, prefill, 512 B/cycle. The 511× is a MAC-count artifact; the 65× reflects score-matrix DRAM elimination.

T	vs 64×64 unfused	vs MAC-norm unfused
512	31.9×	5.0×
2,048	127.8×	17.0×
8,192	511.0×	64.9×

The 64×64 baseline is perpetually compute-bound (7.8× at T=8,192), making the 511× a MAC-count artifact. Equalizing MACs flips it memory-bound, paying 17.45 GB of score-matrix DRAM (98.4% of its traffic); both KV-stat and FlashAttention eliminate this, explaining the 65× over the MAC-matched unfused baseline. Against the fused baseline — equal MACs, equal DRAM — the ratio collapses to ∼1×, confirming DRAM-bound operation.

3.5 Physical Cost

Table 4 shows the silicon cost behind the raw speedup numbers.

Table 4: Physical resource requirements for the best prefill configuration (n=0, lmc=16). PE count and SRAM scale linearly with T.

Resource	T=8,192	T=2,048
Total PEs	524,288	131,072
Total MACs	8,388,608	2,097,152
On-chip K-buffer SRAM	128 MB	34 MB
Estimated die area	∼20 mm^2	∼5 mm^2

The 128 MB K-buffer at T=8,192 is prohibitive for edge targets; at the validated T=2,048 point, 34 MB SRAM and 131K PEs are tractable but still 32× more PEs than the 64×64 FlashAttention reference at equal cycle count. PE count and SRAM scale linearly with T, motivating the multi-pass silicon re-use scheme (Appendix A). An energy estimation using Accelergy with CACTI-based component models was attempted; preliminary component-level breakdowns suggest the primary saving comes from eliminating the score-matrix DRAM round-trip (which dominates unfused-baseline energy), but a validated end-to-end figure was not obtained.

4 Conclusions

What worked. Three techniques combine to improve attention throughput. *(1) KV-stationary dataflow with inline online softmax:* keys and values remain resident in PE columns while queries stream through, computing the softmax update $m'=\max(m, s_i)$, $l'=l\,e^{m-m'}+e^{s_i-m'}$, $O'=O\,e^{m-m'}+V_i\,e^{s_i-m'}$ at each column. Scores are consumed immediately and never written to DRAM, eliminating the $H\times T^2$ score matrix and yielding a genuine 65× over any unfused baseline at T=8,192. *(2) Interleaved MACs (lmc):* time-multiplexing lmc independent MAC units per column, each staggered by $\lceil d/lmc \rceil$ cycles, raises upper-PE utilization from 5% to 87.5% with no extra rows. *(3) Merge extensions (n):* stacking 2^n sub-arrays and combining partial softmax states via a binary merge tree shortens pipeline drain from $T-1$ to $T/2^n-1$ column traversals. The hybrid SCALE-Sim/analytical methodology confirmed wavefront pipelining at ∼2.5 cycles/column and established the regime boundary.

Compute-bound results. When bandwidth is not the bottleneck — achieved with HBM-class memory or by reducing lmc to the balance point — the full benefit of all three techniques is realised. In the compute-only limit (Table 2): n=3, lmc=16 reaches **1.76**× per MAC over FlashAttention at T=8,192, because the taller, narrower sub-array reduces the pipeline traversal from 8,192 to 1,024 columns. n=0, lmc=16 reaches 1.04× per MAC — the flat array offers no drain savings, so the benefit is marginal. The 1.76× ceiling represents the maximum per-silicon advantage the KV-stationary approach can deliver over a fused baseline.

What did not work. At commodity 512 B/cycle, maximizing lmc past the balance point ($lmc \approx$ BW/128) leaves 75% of MACs idle; the efficient choice is $lmc \leq 4$, not lmc=16. Decode validation also failed: the analytical model counts d MAC cycles per step while SCALE-Sim measures the full T-column pipeline traversal, producing divergence of thousands of percent.

Conclusion. The KV-stationary dataflow eliminates score-matrix DRAM by construction and delivers up to **1.76**× per-MAC advantage over fused FlashAttention in the compute-bound regime using merge extensions and interleaved MACs. At commodity 512 B/cycle both architectures are DRAM-bound with identical traffic, collapsing the advantage to ∼1× — the memory wall, not compute efficiency, is the binding constraint. The actionable design rules are: size lmc to bandwidth ($lmc \approx \mathrm{BW}/128$); deploy merge extensions only when compute-bound; and use the multi-pass re-use model to reduce on-chip SRAM by $P\times$ at a sub-linear latency cost (Appendix A).

5 Team Contributions and Acknowledgement

Task Allocation

Component	Completed by
Architecture conception & design iteration	Hudson
Analytical pipeline model (`kv_stationary_model.py`)	Hudson
Baseline roofline model (`baseline_mqa_model.py`)	Dhruv
SCALE-Sim validation harness	Dhruv
Architecture diagrams & visualizations	Hudson
Wavefront fill analysis & BW co-design	Dhruv
Sweep scripts & result CSV generation	Hudson
Figure-plotting scripts	Dhruv
DRAM traffic analysis & energy estimation	Hudson
Report writing & editing	Both

Contributions

Hudson Strauss — ∼50%. Led analytical model development, SCALE-Sim validation pipeline, wavefront fill discovery, bandwidth co-design analysis, and primary report structure.

Dhruv Patel — ∼50%. Led baseline model implementation, architecture diagrams, DRAM traffic and energy analysis, result figure generation, and report review.

References

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training generalized multi-query transformer models from multi-head checkpoints. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 4895–4901. https://aclanthology.org/2023.emnlp-main.298/

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems*, *35*.

Ham, T. J., Jung, S. J., Kim, S., Oh, Y. H., Park, Y., Song, Y., Park, J.-H., Lee, J., Deaton, D., Kim, T. H., et al. (2020). A^3: Accelerating attention mechanisms in neural networks with approximation. *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*.

Samajdar, A., Joseph, J. M., Zhu, Y., Whatmough, P., Mattina, M., & Krishna, T. (2020). A systematic methodology for characterizing scalability of DNN accelerators using SCALE-Sim. *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 58–68.

Wang, H., Zhang, Z., & Han, S. (2021). SpAtten: Efficient sparse attention architecture with cascade token and head pruning. *2021 IEEE International Symposium on High Performance Computer Architecture (HPCA)*.

A Future Work: Silicon Re-Use and Two-Pass Decode

The primary barrier to practical decode deployment is on-chip SRAM. At T=8,192 the full KV cache requires 128 MB — prohibitive for edge or embedded targets. A **multi-pass silicon re-use scheme** addresses this directly.

Concept. Rather than holding the entire KV cache on-chip across all tokens, the sequence is partitioned into P equal slices of T/P tokens each. Each decode step makes P passes over the same physical array (now sized $H \times T/P$ columns), accumulating the running softmax state (m, l, O) across passes using the same online-softmax merge formula used by the merge tree for prefill. After all P passes the final output is produced identically to the single-pass case.

Hardware cost. The array itself is unchanged. At each pass boundary, the partial state (m, l, O) for each active head — $(d + 2)$ elements = 260 bytes at FP16 — is written to a small register and re-injected via a single merge PE (Figure 6) at the start of the next pass. This re-injection costs $2\tau_{\text{exp}} + \lceil 6d/w \rceil = 8{+}6 = 14$ cycles per pass boundary, negligible relative to single-pass cycles ($\sim$1M at T=8,192).

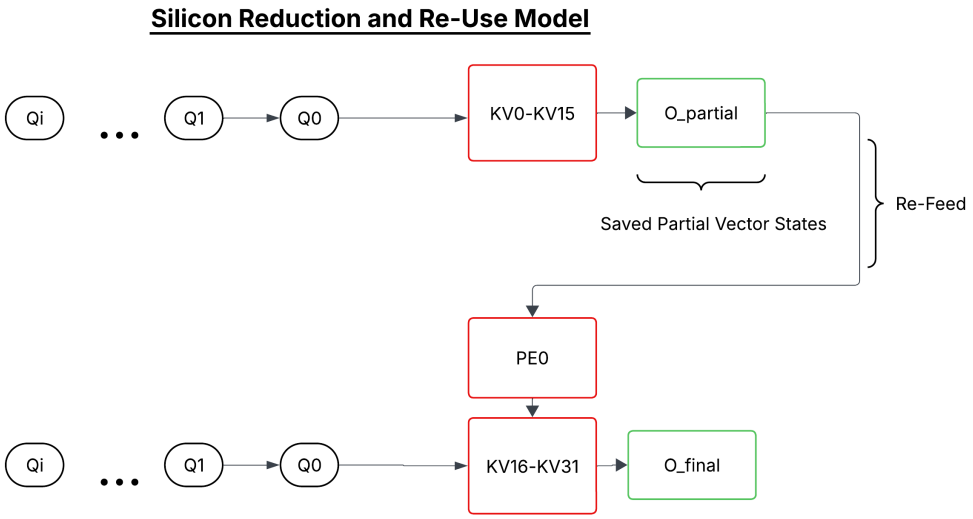


Figure 3: **Silicon re-use model (P=2 passes).** Pass 1: the query stream attends over the first KV slice, producing partial state `O_partial` that is saved. Pass 2: the same query re-enters the second slice; the saved state is re-injected so the online softmax continues from the correct running maximum. `O_final` is the correctly normalised output over all T tokens. The array uses half the columns (and half the SRAM) of a single-pass design.

Silicon reduction and the latency trade-off.

Passes P	Array cols	On-chip SRAM	Cycle multiplier	Applies to
1	T	128 MB	1×	prefill + decode
2	$T/2$	64 MB	≈2× + 14 cyc	prefill + decode
4	$T/4$	32 MB	≈4× + 42 cyc	prefill + decode
8	$T/8$	16 MB	≈8× + 98 cyc	prefill + decode

The latency overhead is sub-linear in P at large T: the P=1 pipeline at T=8,192 is already 99% drain, so splitting into P smaller passes costs only 1.02–1.11× more total cycles while delivering P× smaller chip and P× less SRAM. With P=4 the T=8,192 problem maps exactly to the validated T=2,048 operating point (64×2048 array), running in ≈4× more cycles with 4× less silicon. For causal prefill, the savings are even larger: finished queries are discarded at each pass boundary, reducing Q-read DRAM by up to 69% at P=16 (from the 750-row sweep in `reuse_full_sweep.csv`).

Open questions. (1) SCALE-Sim cycle validation of a single-pass decode step. With query_tokens=1 the analytical model counts only d=128 MAC cycles, but SCALE-Sim measures the full pipeline traversal: a single Q packet must ripple through all T columns and drain out, costing ≈$(R+C-1)\times 2$ cycles (≈16,500 cycles at T=8,192, error ≈130×). Fixing this requires a traversal-time model rather than a MAC-count model for decode. (2) Extending the analytical model to multi-pass decode with full inter-pass softmax merge cost. (3) Optimal P balancing SRAM budget against acceptable decode latency for the target workload.

B Architecture Diagrams

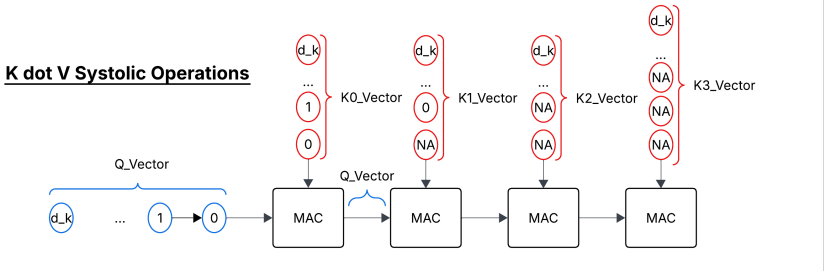
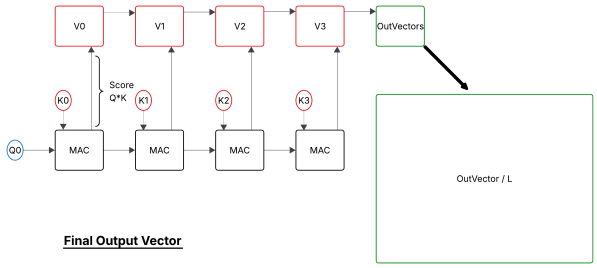
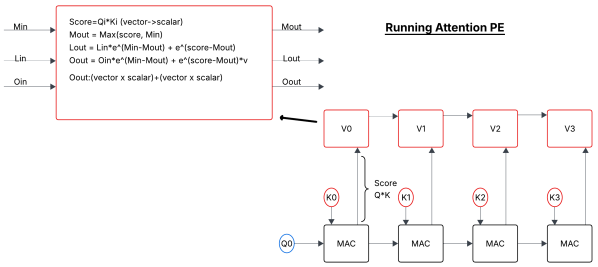


Figure 4: Single-row 1D KV-stationary baseline (Iteration 1). One query vector Q streams left-to-right through T columns; each column holds a stationary (K_i, V_i) pair. This is the starting point before multi-head extension, merge extensions, and interleaved MACs were added.



(a) Single-row dataflow. Query Q_0 streams left-to-right through MAC columns; each column computes score $s_i = Q_0 \cdot K_i$ and feeds the stationary V_i register above. After the final column, `OutVectors` accumulates the weighted sum and normalises by l_N to produce `OutVector / L`.



(b) Running Attention PE detail. The PE receives running state $(M_{\text{in}}, L_{\text{in}}, O_{\text{in}})$ and score $s_i = Q_i \cdot K_i$ (computed below by the MAC array), then updates: $M_{\text{out}} = \max(M_{\text{in}}, s_i)$; $L_{\text{out}} = L_{\text{in}}\, e^{M_{\text{in}} - M_{\text{out}}} + e^{s_i - M_{\text{out}}}$; $O_{\text{out}} = O_{\text{in}}\, e^{M_{\text{in}} - M_{\text{out}}} + e^{s_i - M_{\text{out}}} \cdot V_i$. Updated state $(M_{\text{out}}, L_{\text{out}}, O_{\text{out}})$ passes right.

Figure 5: KV-stationary dataflow (left) and Running Attention PE internals (right). Together these show how one query head streams through T columns, accumulating online softmax state without materialising any score matrix.

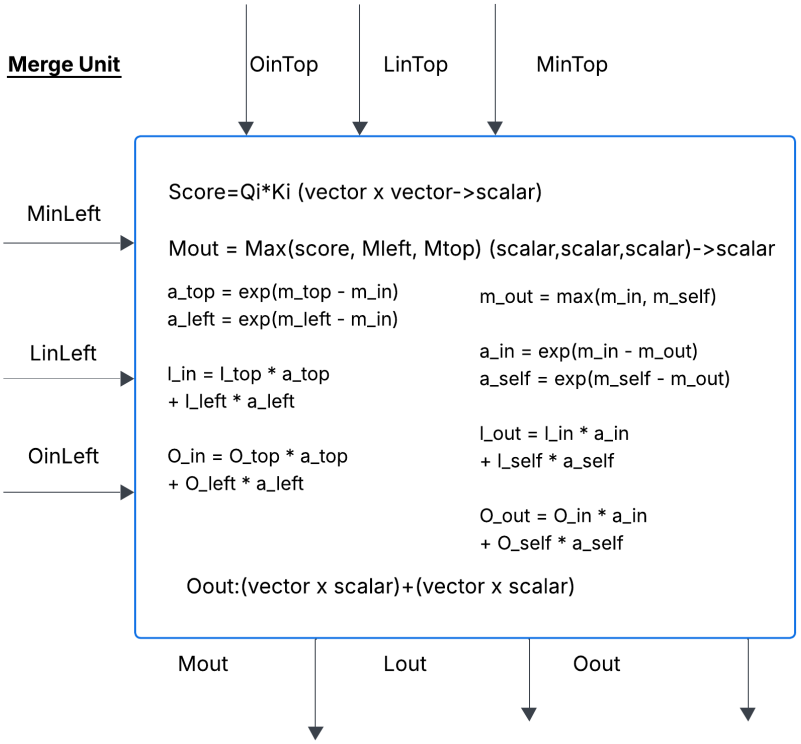


Figure 6: Merge Unit internals. Receives partial softmax states $(M_{\text{left}}, L_{\text{left}}, O_{\text{left}})$ from the row accumulation so far and $(M_{\text{top}}, L_{\text{top}}, O_{\text{top}})$ from the top sub-array (or previous re-use pass). The unit also computes the current-token score $s_i{=}Q_i{\cdot}K_i$, making it a combined attention-and-merge PE: $M_{\text{out}}{=}\max(s_i, M_{\text{left}}, M_{\text{top}})$; $L_{\text{out}}{=}L_{\text{top}}\, e^{M_{\text{top}}-M_{\text{out}}} + L_{\text{left}}\, e^{M_{\text{left}}-M_{\text{out}}}$; $O_{\text{out}}{=}O_{\text{top}}\, e^{M_{\text{top}}-M_{\text{out}}} + O_{\text{left}}\, e^{M_{\text{left}}-M_{\text{out}}}$. Note: when used purely for cross-sub-array merging (no new token), s_i is omitted and the unit reduces to a two-way state combiner. Cost: $2\tau_{\exp} + \lceil 6d/w \rceil = 14$ cycles.

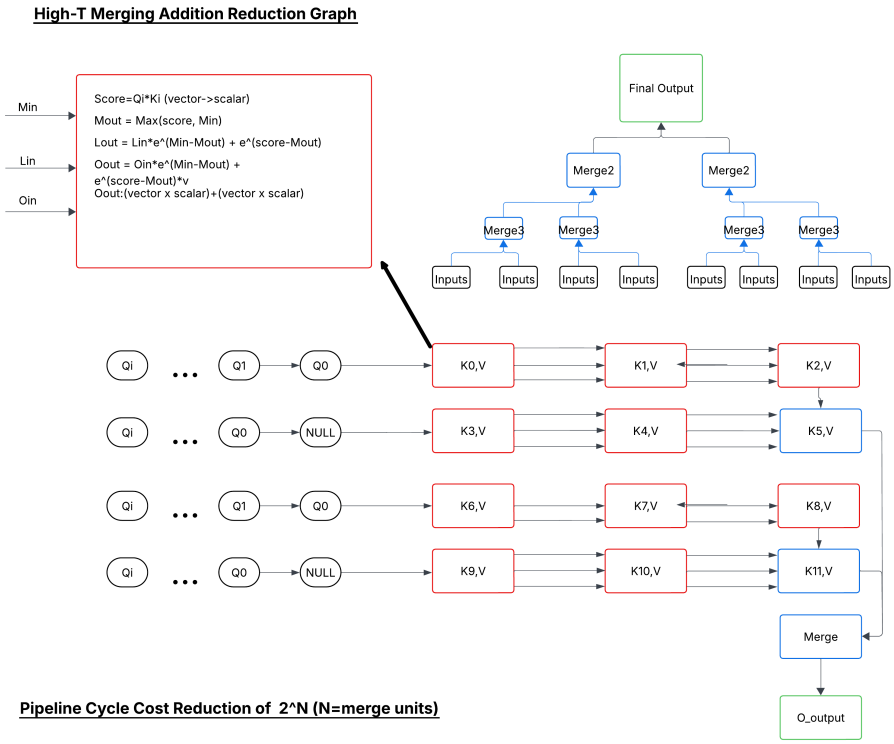


Figure 7: Merge-extension binary reduction tree (n=3, eight sub-arrays). Each leaf sub-array covers $T/2^n$ KV columns and produces a partial (M, L, O) state. Merge Units at each tree level fuse pairs of partial states using the equations in Figure 6, halving the number of active states per level until a single output state remains. Tree depth is n levels, adding $n \times 14$ cycles of merge latency. The same structure is reused at re-use pass boundaries to combine the previous-pass state with the current-pass output.

C Additional Data: Compute-Bound Prefill Results

The figures below show KV-stationary prefill performance in the compute-bound regime (infinite bandwidth / SCALE-Sim compute isolation), where the DRAM floor is removed and pipeline efficiency is the binding constraint. This corresponds to HBM-class bandwidth deployments or the theoretical ceiling of each design. All results use H=64, d=128, FP16, and compare against an equal-area fused FlashAttention baseline ($H \times T$ total PEs on both sides).

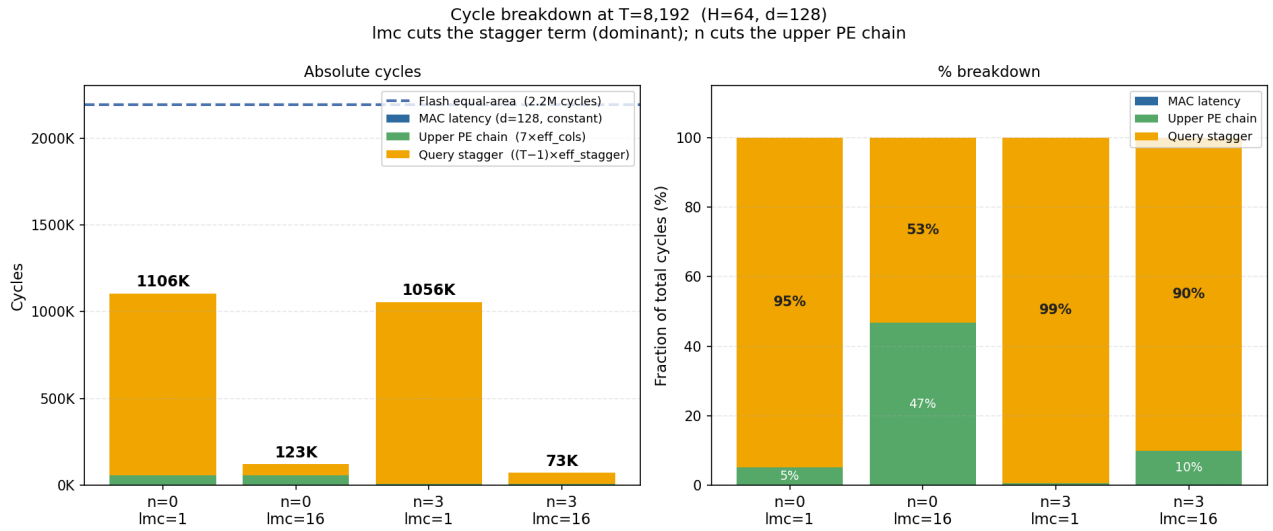


Figure 8: Cycle composition at T=8,192 for all four configurations. lmc=16 cuts eff_stagger from 128 to 8 cycles, reducing total cycles from 1,106K to 123K (n=0) and from 1,056K to 73K (n=3). The query-stagger term (orange) is dominant for lmc=1 ($\geq$95%) but only 53% for n=0, lmc=16 and 90% for n=3, lmc=16 — once stagger is suppressed by lmc, the upper-PE chain (green, 47% and 10% respectively) becomes the next bottleneck.

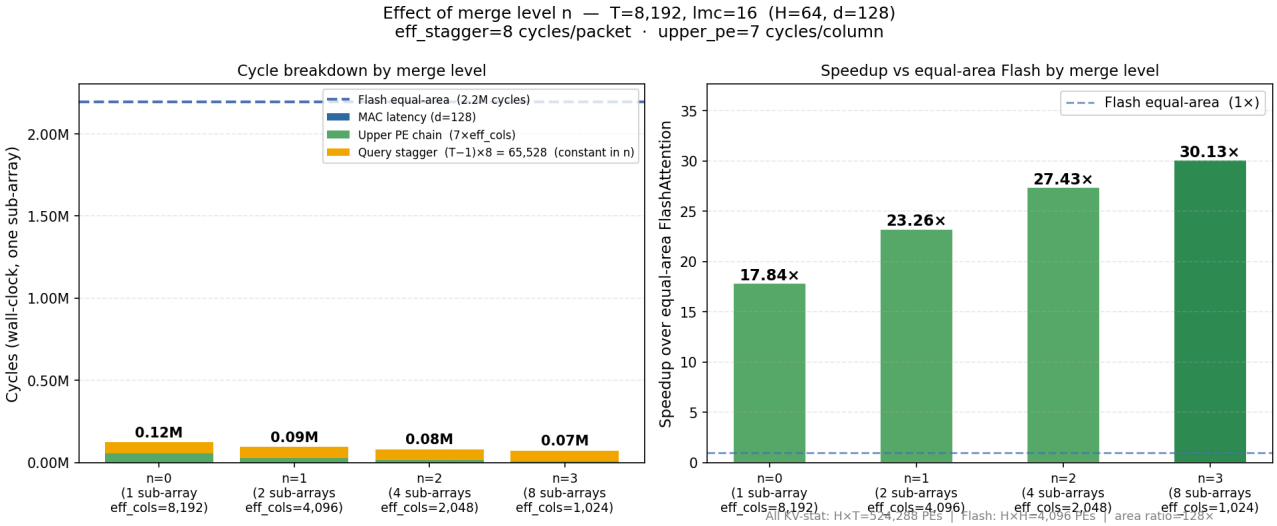


Figure 9: Effect of merge level n at T=8,192, lmc=16 (compute-bound). Left: absolute cycles fall from 0.12M (n=0) to 0.07M (n=3) as the upper-PE chain shortens with sub-array column count. The query-stagger term $(T-1)\times 8$ is constant in n (orange); only the upper-PE chain (green, 7×eff_cols) shrinks. Right: speedup over equal-area FlashAttention (both sides: $H\times T$ total PEs; Flash equal-area = flash_total / (T/H)) rises from 17.8× (n=0) to **30.1**× (n=3). Note that KV-stat performs $4d$ MACs per token pair vs $2d$ for Flash; the per-MAC ceiling is ≈15× at n=3.

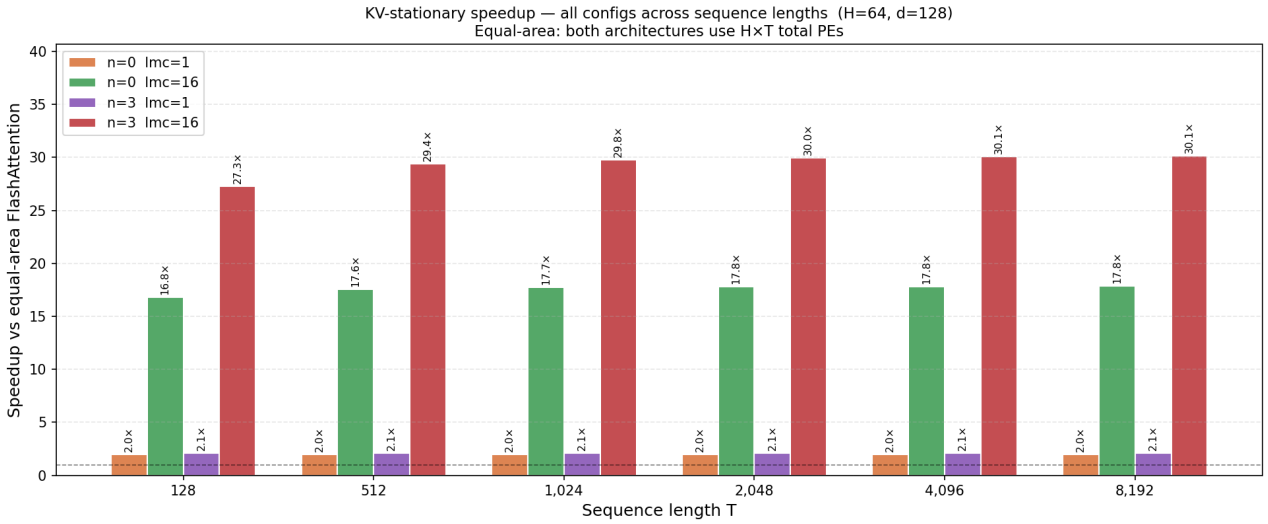
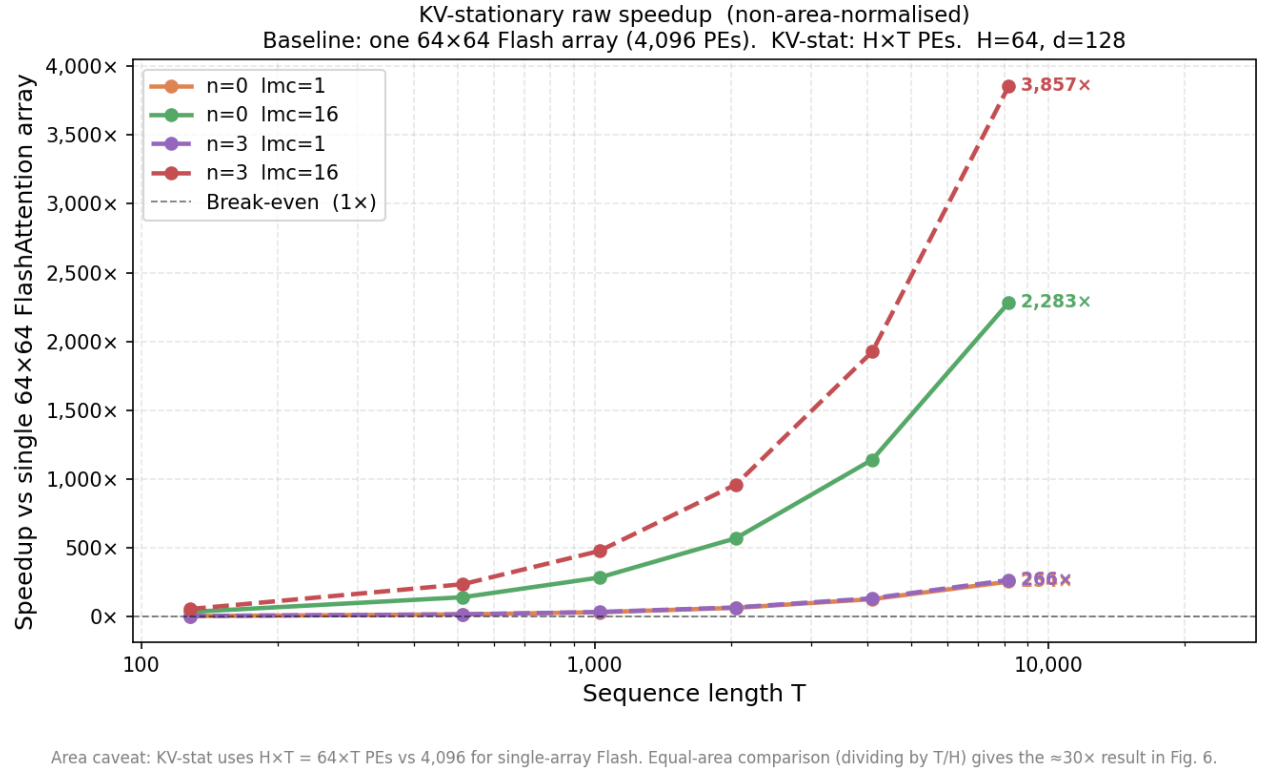


Figure 10: *Raw* compute-only speedup over equal-area FlashAttention (equal-area: KV-stat and Flash each given $H \times T$ PEs; no DRAM floor applied). n=3, lmc=16 (red) saturates at ≈30× for $T \geq 512$; n=0, lmc=16 (green) at ≈18×. The flat profile confirms the speedup is pipeline-limited (stagger + upper-PE terms), not sequence-length-limited. lmc=1 configurations deliver only 2× regardless of n. **Note:** this is not the same metric as Table 2 (§3). Table 2 normalises by MAC count; since KV-stat performs $4d$ MACs per token pair vs $2d$ for Flash (a ≈16× MAC ratio at lmc=16), the per-MAC ceiling is $30 \times /16 \approx 1.9\times$, consistent with Table 2's 1.76×.



Area caveat: KV-stat uses H×T = 64×T PEs vs 4,096 for single-array Flash. Equal-area comparison (dividing by T/H) gives the ≈30× result in Fig. 6.

Figure 11: Raw (non-area-normalised) compute-only speedup over a *single* $H \times H = 64 \times 64$ FlashAttention array. Unlike Figure 10, no area equalization is applied: KV-stat uses $H \times T$ PEs while Flash uses H^2=4,096 PEs. Speedup grows $\propto T$ because Flash compute is $O(T^2)$ while the KV-stationary pipeline is $O(T)$. n=3, lmc=16 reaches 3,857× at T=8,192; n=0, lmc=16 reaches 2,283×. Dividing by the area ratio T/H (128 at T=8,192) recovers the ≈30× equal-area speedup of Figure 10.